

PROJECT: E2E AWS Pipeline to Analyze Bitcoin (BTC) and Monero (XMR) Price with Google Trends Sentiment (2019–2024).

PURPOSE: We analyze the correlation between Google search trend and the price volatility of a cryptocurrency. We assess the strength of this correlation and determine which of the two factors - price movements or online interest - drives the relationship, using data from Bitcoin (BTC) and Monero (XMR) over the period 2019–2024.

Through a cloud environment, large volumes of data can be leveraged, enabling full automation of the process - from raw data collection to the availability of strategic insights, via function orchestration systems. Another key aspect is the scalability of the framework, which allows for parallel processing and the possibility of extending the analysis to other cryptocurrencies and macro analysis factors.

METHOD AND AWS SERVICES:

- **S3:** We store four CSV files: BTC_History_Data, BTC_Google_Trends, XMR_History_Data, XMR_Google_Trends, which contain the price trends of Bitcoin and Monero and their respective Google search sentiment trends. These first storage environment is called "bucketraw".
- **Glue ETL – Data Preparation and Unification:** The Price and Trends files are cleaned and aligned in terms of time period and measurement frequency. The price is calculated as a weekly average to match the time frame used in online search measurements and to reduce "noise" caused by daily price spikes. The cleaned file is saved in Parquet format in "bucketargent".
- **Redshift – Data Analysis:** The two tables, each corresponding to a cryptocurrency, are merged into a single Parquet file containing 'Date' – 'Price' – 'GoogleTrendIndex'. This file is stored in "bucketgolden" and loaded into a Redshift table to enable further analysis in other environments.
- **Step Functions – Process Orchestration:** The Glue ETL transformations are integrated into a pipeline. The two separate pipelines for each cryptocurrency can run in parallel, maximizing process efficiency.
- **QuickSight – Data Visualization and Insights:** The Bitcoin and Monero tables in Redshift are ready for queries and additional processing. They are visualized through QuickSight to immediately capture the correlation between the two trends.

Bitcoin and Monero CSV Files:

- **History_Data:** Columns: "Date" – Date of the recorded value, "Price" – Daily closing price, "Open" – Daily opening price, "High" – Highest price of the day, "Low" – Lowest price of the day, "Vol." – Daily trading volume, "Change %" – Daily percentage change. We use the first two columns and clean the Price column by removing incorrect values of -1.
- **Google Trends:** Columns: "Date" – Measured in weeks, and "Cryptocurrency Interest" – A measure of the search trend for the cryptocurrency on Google, ranging from 0 to 100.

```
"Date","Price","Open","High","Low","Vol. ","Change %"  
"03/12/2024","65,619.5","65,860.0","66,135.3","65,200.0","0.19K","-0.37%"  
"03/11/2024","65,859.9","63,074.2","66,600.0","61,517.2","1.88K","4.42%"  
"03/10/2024","63,074.3","62,613.1","63,930.6","62,400.0","0.72K","0.68%"  
"03/09/2024","62,649.9","62,425.2","62,781.1","62,124.5","0.37K","0.38%"  
"03/08/2024","62,413.8","61,130.1","63,999.0","60,483.8","1.61K","2.10%"  
"03/07/2024","61,130.1","60,642.5","62,364.9","60,201.1","1.13K","0.82%"  
"03/06/2024","60,633.8","58,788.1","62,227.0","57,937.5","1.76K","3.06%"  
"03/05/2024","58,830.7","62,877.2","63,584.5","54,425.1","3.94K","-6.51%"  
"03/04/2024","62,930.5","58,214.0","63,089.9","57,580.2","3.39K","8.07%"  
"03/03/2024","58,230.2","57,223.2","58,300.0","56,650.1","0.63K","1.76%"  
"03/02/2024","57,223.2","57,564.8","57,608.4","56,900.0","0.40K","-0.59%"
```

```
"Settimana","interesse bitcoin"  
2019-03-17,9  
2019-03-24,9  
2019-03-31,15  
2019-04-07,13  
2019-04-14,11
```

Dataset Preparation

Bucket per uso generico

Bucket di director

Bucket per uso generico (3) Info

Tutte le Re

I bucket sono container per i dati archiviati in S3.

Cerca bucket in base al nome

	Nome
☐	bucketargent
☐	bucketgolden
☐	bucketraw

bucketraw Info

Oggetti

Proprietà

Autorizzazioni

Parametri

Oggetti (4)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ott
necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori info](#)

Trova oggetti per prefisso

☐	Nome	Tipo
☐	BTC_EUR_Historical_Data.csv	csv
☐	google_trend_bitcoin.csv	csv
☐	google_trend_monero.csv	csv
☐	XMR_EUR Kraken Historical Data.csv	csv

Glue ETL - Notebook

"BTC Files Review"

Visualization and analysis for the subsequent processing of the "BTC History Data" file using a Glue Script:

AWS Glue Studio Notebook

You are now running a AWS Glue Studio notebook; To start using your notebook you

Check and transformation of the Bitcoin History Data File

```
from aws glue.context import GlueContext
from pyspark.context import SparkContext
from pyspark.sql import functions as F

glueContext = GlueContext(SparkContext.getOrCreate())
spark = glueContext.spark_session

# Defines the S3 paths
input_s3_path = "s3://bucketraw/BTC_EUR_Historical_Data.csv"
output_s3_path = "s3://bucketraw/Sorted_BTC_EUR_Historical_Data.csv"

# Reads the CSV file from S3 as a DataFrame
df = spark.read.csv(input_s3_path, header=True, inferSchema=True)

# Converts the "Date" column to date type
df = df.withColumn("Date", F.to_date(F.col("Date"), "MM/dd/yyyy"))

# Sorts the DataFrame by the "Date" column in ascending order
df_sorted = df.orderBy("Date")

# Displays a preview of the sorted table
df_sorted.show(20, truncate=False)

# Writes the sorted DataFrame back as CSV to S3
df_sorted.write.csv(output_s3_path, header=True, mode="overwrite")

# Filters to display only the columns "Date" and "Price"
print("Data types of Date and Price:")
for col_name, col_type in df.dtypes:
    if col_name in ["Date", "Price"]:
        print(f"{col_name}: {col_type}")
```

Date	Price	Open	High	Low	Vol.	Change %
2019-01-12	3,168.1	3,175.0	3,193.2	3,130.0	2.36K	-0.21%
2019-01-13	3,074.0	3,168.1	3,188.0	3,052.2	3.52K	-2.97%
2019-01-14	3,197.6	3,075.0	3,238.8	3,068.1	7.29K	4.02%
2019-01-15	3,149.2	3,197.6	3,214.3	3,119.8	5.36K	-1.51%
2019-01-16	3,164.7	3,149.2	3,228.9	3,140.2	5.52K	0.49%
2019-01-17	3,196.5	3,167.8	3,213.3	3,123.7	6.44K	1.00%
2019-01-18	3,179.9	3,196.5	3,200.0	3,156.9	4.25K	-0.52%
2019-01-19	3,245.3	3,179.9	3,311.6	3,177.1	3.59K	2.06%
2019-01-20	3,110.4	3,245.3	3,262.3	3,062.0	4.03K	-4.16%
2019-01-21	3,112.7	3,113.8	3,148.7	3,077.4	4.53K	0.07%
2019-01-22	3,149.4	3,112.7	3,175.0	3,032.2	6.45K	1.18%
2019-01-23	3,126.6	3,149.4	3,188.1	3,102.6	5.18K	-0.72%
2019-01-24	3,158.7	3,126.6	3,181.4	3,104.8	3.53K	1.03%
2019-01-25	3,126.4	3,158.7	3,161.1	3,102.3	4.29K	-1.02%
2019-01-26	3,126.0	3,126.5	3,206.9	3,112.6	2.43K	-0.01%
2019-01-27	3,096.6	3,126.0	3,130.9	3,051.0	2.22K	-0.94%
2019-01-28	3,003.3	3,096.6	3,099.7	2,957.3	6.47K	-3.01%
2019-01-29	2,973.4	3,003.3	3,011.5	2,927.2	4.61K	-1.00%
2019-01-30	2,999.9	2,973.4	3,030.3	2,957.4	4.37K	0.89%
2019-01-31	2,978.7	2,999.9	3,025.0	2,965.3	3.88K	-0.71%

only showing top 20 rows

```

: # Filters to display only the columns "Date" and "Price"
print("Data types of Date and Price:")
for col_name, col_type in df.dtypes:
    if col_name in ["Date", "Price"]:
        print(f"{col_name}: {col_type}")

Data types of Date and Price:
Date: date
Price: string

: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lag, when
from pyspark.sql.window import Window

# Keeps only the columns "Date" and "Price"
df_transformed = df_sorted.drop("Open", "High", "Low", "Change %", "Vol.", "Index")

# Defines the window to access the previous value
window_spec = Window.orderBy("Date")

# Replaces -1 values with the previous day's price
df_transformed = df_transformed.withColumn(
    "Price",
    when(col("Price") == -1, lag("Price").over(window_spec)).otherwise(col("Price"))
)

# Displays the first rows of the transformed DataFrame
df_transformed.show(20)

```

```

+-----+-----+
|      Date| Price|
+-----+-----+
|2019-01-12|3,168.1|
|2019-01-13|3,074.0|
|2019-01-14|3,197.6|
|2019-01-15|3,149.2|
|2019-01-16|3,164.7|
|2019-01-17|3,196.5|
|2019-01-18|3,179.9|
|2019-01-19|3,245.3|
|2019-01-20|3,110.4|
|2019-01-21|3,112.7|
|2019-01-22|3,149.4|
|2019-01-23|3,126.6|
|2019-01-24|3,158.7|
|2019-01-25|3,126.4|
|2019-01-26|3,126.0|
|2019-01-27|3,096.6|
|2019-01-28|3,003.3|
|2019-01-29|2,973.4|
|2019-01-30|2,999.9|
|2019-01-31|2,978.7|
+-----+-----+
only showing top 20 rows

```

```

from pyspark.sql import functions as F

# Removes the comma (thousands separator) and keeps the decimal point unchanged
df_transformed = df_transformed.withColumn(
    "Price",
    F.regexp_replace(F.col("Price"), ",", "")
)

# Converts the "Price" column to float
df_transformed = df_transformed.withColumn(
    "Price",
    F.col("Price").cast("float")
)

# Displays the first 5 results
df_transformed.show(5, truncate=False)

# Checks the data type of the "Price" column
print("Data type of the 'Price' column:")
for col_name, col_type in df_transformed.dtypes:
    if col_name == "Price":
        print(f"{col_name}: {col_type}")

```

```

+-----+-----+
|Date      |Price |
+-----+-----+
|2019-01-12|3168.1|
|2019-01-13|3074.0|
|2019-01-14|3197.6|
|2019-01-15|3149.2|
|2019-01-16|3164.7|
+-----+-----+
only showing top 5 rows

```

Tipo di dato della colonna 'Price':
Price: float

```

: # Truncates the decimals after the point
df_transformed = df_transformed.withColumn(
    "Price",
    F.floor(F.col("Price"))
)

# Replaces the comma with a point where it was previously present
df_transformed = df_transformed.withColumn(
    "Price",
    F.format_number(F.col("Price"), 0) # Format as integer, no decimals
)

# Displays the results
df_transformed.show(10)

```

```

+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-12| null|
|2019-01-13| null|
|2019-01-14| null|
|2019-01-15| null|
|2019-01-16| null|
|2019-01-17| null|
|2019-01-18| null|
|2019-01-19| null|
|2019-01-20| null|
|2019-01-21| null|
+-----+-----+
only showing top 10 rows

```

```
from pyspark.sql.functions import col, regexp_replace

# Converts the Price column to string, replaces the comma with a point
df_transformed = df_transformed.withColumn(
    "Price", regexp_replace(col("Price").cast("string"), ",", ".")
)

# Converts Price to float type for future calculations
df_transformed = df_transformed.withColumn(
    "Price", col("Price").cast("float")
)

# Displays the result
df_transformed.show(10)
```

```
+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-12|3.168|
|2019-01-13|3.074|
|2019-01-14|3.197|
|2019-01-15|3.149|
|2019-01-16|3.164|
|2019-01-17|3.196|
|2019-01-18|3.179|
|2019-01-19|3.245|
|2019-01-20| 3.11|
|2019-01-21|3.112|
+-----+-----+
only showing top 10 rows
```

```
# Displays the first 5 rows
print("First 5 rows:")
df_transformed.show(5)

# Displays the Last 5 rows
print("Last 5 rows:")
df_transformed.orderBy(col("Date").desc()).show(5)

# Counts the total number of rows
num_rows = df_transformed.count()
print(f"Total number of rows: {num_rows}")
```

```
First 5 rows:
+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-12| null|
|2019-01-13| null|
|2019-01-14| null|
|2019-01-15| null|
|2019-01-16| null|
+-----+-----+
only showing top 5 rows
```

```
Last 5 rows:
+-----+-----+
|      Date|Price|
+-----+-----+
|2024-03-12| null|
|2024-03-11| null|
|2024-03-10| null|
|2024-03-09| null|
|2024-03-08| null|
+-----+-----+
only showing top 5 rows
```

Verification of the BTC Google Trends File:

```
input_s3_path = "s3://bucketraw/google_trend_bitcoin.csv"
output_s3_path = "s3://bucketargent/Sorted_google_trend_bitcoin.csv"

# Reads the CSV file from S3 as a DataFrame
df_btccgl = spark.read.csv(input_s3_path, header=True, inferSchema=True)

# Lists the column names of the DataFrame
print("Column names:", df_btccgl.columns)

Nomi delle colonne: ['Settimana', 'interesse bitcoin']
```

```
"Settimana": Week
"interesse bitcoin": Interest in Bitcoin

# Converts the Week ("Settimana") column to date type
df_btccgl = df_btccgl.withColumn("Settimana", F.to_date(F.col("Settimana"), "MM/dd/yyyy"))

# Sorts the DataFrame by the "Date" column in ascending order
df_btccgl_sorted = df_btccgl.orderBy("Settimana")

# Displays the first 5 rows of the sorted DataFrame
df_btccgl_sorted.show(5)
```

```
+-----+-----+
| Settimana|interesse bitcoin|
+-----+-----+
|2019-03-17|          9|
|2019-03-24|          9|
|2019-03-31|         15|
|2019-04-07|         13|
|2019-04-14|         11|
+-----+-----+
only showing top 5 rows
```

```
# Saves the DataFrame as a CSV on S3
df_btccgl_sorted.write.csv(output_s3_path, mode="overwrite", header=True)
```

```
# Displays the first 5 rows
print("First 5 rows:")
df_btccgl_sorted.show(5)

# Displays the Last 5 rows
print("Last 5 rows:")
df_btccgl_sorted.orderBy(col("Settimana").desc()).show(5)

# Counts the total number of rows
num_rows = df_btccgl_sorted.count()
print(f"Total number of rows: {num_rows}")
```

```
First 5 rows:
+-----+-----+
| Settimana|interesse bitcoin|
+-----+-----+
|2019-03-17|          9|
|2019-03-24|          9|
|2019-03-31|         15|
|2019-04-07|         13|
|2019-04-14|         11|
+-----+-----+
only showing top 5 rows
```

```
Last 5 rows:
+-----+-----+
| Settimana|interesse bitcoin|
+-----+-----+
|2024-03-03|         52|
|2024-02-25|         43|
|2024-02-18|         22|
|2024-02-11|         29|
|2024-02-04|         20|
+-----+-----+
only showing top 5 rows
Total number of rows: 260
```

Alignment of "BTC Historical Data" and "BTC Google Trends" by calculating the average price for the weeks defined in "Google Trends", from 17-03-2019 to 03-03-2024:

```
from pyspark.sql.functions import col, min, max, date_add, date_sub, avg, lit, dayofweek, floor

# Determines the observation period from the trend dataset
min_date = df_btccgl_sorted.select(min("Settimana")).first()[0] # "2019-03-17"
max_date = df_btccgl_sorted.select(max("Settimana")).first()[0] # "2024-03-03"

# Filters the Bitcoin dataset for the period defined by the trends
df_btc_filtered = df_transformed.filter(
    (col("Date") >= lit(min_date)) & (col("Date") <= lit(max_date))
)

# Checks for any null values in the "Price" column
df_btc_filtered = df_btc_filtered.filter(col("Price").isNotNull())

# Calculates the start date of the week (opening day: Sunday)
df_btc_aligned = df_btc_filtered.withColumn(
    "Week_Start",
    date_sub(col("Date"), (dayofweek(col("Date")) - 1))
)

# Groups by correct weeks and calculates the weekly average price
df_btc_weekly = df_btc_aligned.groupBy("Week_Start").agg(
    avg(floor(col("Price").cast("double"))).alias("Weekly_Avg_Price")
)

# Sorts the weeks
df_btc_weekly = df_btc_weekly.orderBy("Week_Start", ascending=True)

# Displays the first rows for verification
df_btc_weekly.show(10)
```

```
+-----+-----+
|Week_Start| Weekly_Avg_Price|
+-----+-----+
|2019-03-17|                3.0|
|2019-03-24|                3.0|
|2019-03-31| 3.7142857142857144|
|2019-04-07|                4.0|
|2019-04-14|                4.0|
|2019-04-21|                4.0|
|2019-04-28| 4.285714285714286|
|2019-05-05| 5.142857142857143|
|2019-05-12| 6.428571428571429|
|2019-05-19| 6.857142857142857|
+-----+-----+
only showing top 10 rows
```

The same operation is performed for Monero, adjusting the Price column in a slightly different way to be compatible with the reported data type:

```
: from pyspark.sql import functions as F

# Removes the comma (thousands separator) and keeps the decimal point unchanged
df_transformed = df_transformed.withColumn(
    "Price",
    F.regexp_replace(F.col("Price"), ",", "")
)

# Converts the "Price" column to float
df_transformed = df_transformed.withColumn(
    "Price",
    F.col("Price").cast("float")
)

# Displays the first 5 results to see how the data looks
df_transformed.show(5, truncate=False)

# Checks the data type of the "Price" column
print("Data type of the 'Price' column:")
for col_name, col_type in df_transformed.dtypes:
    if col_name == "Price":
        print(f"{col_name}: {col_type}")
```

```
+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-01|41.92|
|2019-01-02|45.76|
|2019-01-03|43.32|
|2019-01-04|43.98|
|2019-01-05|43.43|
|2019-01-06|48.02|
|2019-01-07|45.92|
|2019-01-08|46.17|
|2019-01-09|45.22|
|2019-01-10|39.12|
|2019-01-11|39.37|
|2019-01-12| 38.9|
|2019-01-13|37.18|
|2019-01-14|39.72|
|2019-01-15|38.98|
|2019-01-16|40.26|
|2019-01-17|39.76|
|2019-01-18|39.33|
|2019-01-19|40.27|
|2019-01-20|38.27|
+-----+-----+
only showing top 20 rows
```

```
First 5 rows:
+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-01|41.92|
|2019-01-02|45.76|
|2019-01-03|43.32|
|2019-01-04|43.98|
|2019-01-05|43.43|
+-----+-----+
only showing top 5 rows
```

```
Last 5 rows:
+-----+-----+
|      Date| Price|
+-----+-----+
|2024-03-12|133.29|
|2024-03-11|132.69|
|2024-03-10|133.76|
|2024-03-09| 131.4|
|2024-03-08| 134.7|
+-----+-----+
only showing top 5 rows

Total number of rows: 1898
```

After analyzing the CSV files using the Notebook, we process the file with a Script to clean the data and store it for subsequent processing:

Modify BTC Price/Google Trends Files

[Script](#)
[Job details](#)
[Runs](#)
[Data quality](#)
[Schedules](#)
[Version Control](#)

Script

```

1  from awsglue.context import GlueContext
2  from pyspark.context import SparkContext
3  from pyspark.sql import functions as F
4  from pyspark.sql import SparkSession
5  from pyspark.sql.functions import col, lag, when, regexp_replace, min, max, date_sub, avg, lit, dayofweek, floor
6  from pyspark.sql.window import Window
7
8  glueContext = GlueContext(SparkContext.getOrCreate())
9  spark = glueContext.spark_session
10
11 # S3 Paths
12 input_s3_path = "s3://bucketraw/BTC_EUR_Historical_Data.csv"
13 output_s3_path = "s3://bucketargent/Sorted_BTC_EUR_Historical_Data.parquet"
14
15 input_s3_path_trends = "s3://bucketraw/google_trend_bitcoin.csv"
16 output_s3_path_trends = "s3://bucketargent/Sorted_google_trend_bitcoin.parquet"
17
18 # Reads the CSV files from S3
19 df = spark.read.csv(input_s3_path, header=True, inferSchema=True)
20 df_btccgl = spark.read.csv(input_s3_path_trends, header=True, inferSchema=True)
21
22 # Transformations on the BTC historical dataset
23 df = df.withColumn("Date", F.to_date(F.col("Date"), "MM/dd/yyyy")).orderBy("Date")
24 df_transformed = df.drop("Open", "High", "Low", "Change %", "Vol.", "Index")
25
26 # Replaces -1 values with lag
27 window_spec = Window.orderBy("Date")
28 df_transformed = df_transformed.withColumn(
29     "Price", when(col("Price") == -1, lag("Price").over(window_spec)).otherwise(col("Price"))
30 )
31 df_transformed = df_transformed.withColumn("Price", regexp_replace(col("Price"), ",", "").cast("float"))
32
33 # Filters for the observed period
34 df_btccgl = df_btccgl.withColumn("Settimana", F.to_date(F.col("Settimana"), "MM/dd/yyyy")).orderBy("Settimana")
35 min_date = df_btccgl.select(min("Settimana")).first()[0]
36 max_date = df_btccgl.select(max("Settimana")).first()[0]
37
38 df_btc_filtered = df_transformed.filter((col("Date") >= lit(min_date)) & (col("Date") <= lit(max_date)))
39 df_btc_filtered = df_btc_filtered.filter(col("Price").isNotNull())
40
41 # Week calculation
42 df_btc_aligned = df_btc_filtered.withColumn("Week_Start", date_sub(col("Date"), (dayofweek(col("Date")) - 1)))
43 df_btc_weekly = df_btc_aligned.groupBy("Week_Start").agg(avg(floor(col("Price")))).alias("Weekly_Avg_Price")
44 df_btc_weekly = df_btc_weekly.orderBy("Week_Start")
45
46 # Final save in Parquet
47 df_btc_weekly.write.mode("overwrite").parquet(output_s3_path)
48 df_btccgl.write.mode("overwrite").parquet(output_s3_path_trends)
49

```

AWS Glue Studio

[Create job](#)
[Info](#)

Author in a visual interface focused on data flow.

Visual ETL

Author using an interactive code notebook.

Notebook

Author code with a script editor.

Script editor

► **Example jobs**

Create example job

Your jobs (2)

Filter jobs by property

Job name	Type	Created by	Last modified	AWS Glue version
Modify XMR Price/Google Trends Files	Glue ETL	Script	2/11/2025, 4:22:42 PM	5.0
Modify BTC Price/Google Trends Files	Glue ETL	Script	2/11/2025, 2:28:42 PM	5.0

The Parquet files resulting from the cleaning process are saved in a pre-processing bucket called "bucketargent". The files can be viewed as follows for Bitcoin and Monero using a Parquet file reader. We verify the correct calculation of the average price by using the same weeks in both files.

bucketargentInfo

Oggetti

Proprietà

Autorizzazioni

Parametri

Oggetti (5)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti, è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

Nome

Tipo

Cleaned_XMR_EUR Kraken_Historical_Data.parquet/

Cartella

Sorted_BTC_EUR_Historical_Data.parquet/

Cartella

Sorted_google_trend_bitcoin.csv/

Cartella

Sorted_google_trend_bitcoin.parquet/

Cartella

Sorted_google_trend_monero.parquet/

Cartella

Sorted_BTC_EUR_Historical_Data.parquet/

Oggetti

Proprietà

Oggetti (1)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti, è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

Nome

Tipo

part-00000-fe712cb3-1c0b-4972-b7ac-95dbd9040ad3-c000.snappy.parquet

parquet

Cleaned_XMR_EUR Kraken_Historical_Data.parquet/

Oggetti

Proprietà

Oggetti (1)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti, è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

Nome

Tipo

part-00000-fabd68e9-417a-48c7-beeb-46841503aa97-c000.snappy.parquet

parquet

Top 10 Price Trends

Top Selling Weeks

Filters

Add Time

Add Measure

Export

Change data

Week_Start	Weekly_Avg_Price
3/17/2019,...	3,516.286
3/24/2019,...	3,556.571
3/31/2019,...	4,214.143
4/7/2019, ...	4,594
4/14/2019,...	4,629.857
4/21/2019,...	4,745.857
4/28/2019,...	4,816.429
5/5/2019, ...	5,440.714
5/12/2019,...	6,822.286
5/19/2019,...	7,126.571
5/26/2019,...	7,718.714
6/2/2019, ...	7,118.286
6/9/2019, ...	7,296
6/16/2019,...	8,496.429
6/23/2019,...	10,248.571
6/30/2019,...	9,819.571
7/7/2019, ...	10,554
7/14/2019,...	9,229.857
7/21/2019,...	8,932.857
7/28/2019,...	9,051.714
8/4/2019, ...	10,378.857

columns to build your insight

Filters

Add Time

Add Measure

Add Dimension

Week_Start	Weekly_Avg_Price
3/17/2019,...	45.857
3/24/2019,...	46.143
3/31/2019,...	56.857
4/7/2019, ...	60.143
4/14/2019,...	59.714
4/21/2019,...	58
4/28/2019,...	56.429
5/5/2019, ...	60.429
5/12/2019,...	73.714
5/19/2019,...	77.143
5/26/2019,...	83.714
6/2/2019, ...	77.571
6/9/2019, ...	78.857
6/16/2019,...	91
6/23/2019,...	92.714
6/30/2019,...	79
7/7/2019, ...	86.714
7/14/2019,...	73.714
7/21/2019,...	72.286
7/28/2019,...	73.286
8/4/2019, ...	82.571
8/11/2019,...	75.857

7

We create a single file from the two datasets consisting of: "Date" – "Price" – "GoogleTrendsIndex".

Bitcoin (BTC):

BTC_Join_Price_Google trend

Script

Job details

Runs

Data quality

Schedules

Version Control

Script

Info

1

Initialize Glue and Spark

2

glueContext = GlueContext(SparkContext.getOrCreate())

3

spark = glueContext.spark_session

4

5

S3 Paths

6

input_s3_path_btc_weekly = "s3://bucketargent/Sorted_BTC_EUR_Historical_Data.parquet"

7

input_s3_path_trends = "s3://bucketargent/Sorted_google_trend_bitcoin.parquet"

8

output_s3_path_joined = "s3://bucketgolden/BTC_GoogleTrend_Joined.parquet"

9

10

Read Parquet files

11

df_btc_weekly = spark.read.parquet(input_s3_path_btc_weekly)

12

df_btccgl = spark.read.parquet(input_s3_path_trends)

13

14

Select only the necessary columns

15

df_btccgl_selected = df_btccgl.select(

16

col("Settimana").alias("Week_Start"),

17

col("interesse bitcoin").alias("IndiceGoogleTrend")

18

)

19

20

Perform join on weekly fields

21

df_joined = df_btc_weekly.join(df_btccgl_selected, on="Week_Start", how="inner")

22

23

Rename columns for the required format

24

df_final = df_joined.select(

25

col("Week_Start").alias("Data"),

26

col("Weekly_Avg_Price").alias("Prezzo"),

27

col("IndiceGoogleTrend")

28

)

29

30

Save the final result as Parquet

31

df_final.write.mode("overwrite").parquet(output_s3_path_joined)

32

33

print("Parquet successfully created in bucketgolden.")

34

Run status

Retries

Start time (Local)

End time (Local)

Duration

Succeeded

0

02/11/2025 16:52:41

02/11/2025 16:54:25

1 m 36 s

Run details

Input arguments (11)

Continuous logs

Run insights

Metrics

Troubleshooting analysis

Job name

Start time (Local)

Glue version

BTC_Join_Price_Google trend

02/11/2025 16:52:41

5.0

Monero (XMR):

XMR_Join_Price_Google trend

[Script](#) | [Job details](#) | [Runs](#) | [Data quality](#) | [Schedules](#) | [Version Control](#)

Script [Info](#)

```
1  # Initialize Glue and Spark
2  glueContext = GlueContext(SparkContext.getOrCreate())
3  spark = glueContext.spark_session
4
5  # S3 Paths
6  input_s3_path_xmr_weekly = "s3://bucketargent/Cleaned_XMR_EUR_Kraken_Historical_Data.parquet"
7  input_s3_path_trends = "s3://bucketargent/Sorted_google_trend_monero.parquet"
8  output_s3_path_joined = "s3://bucketgolden/XMR_GoogleTrend_Joined.parquet"
9
10 # Read Parquet files
11 # Weekly price dataset for Monero
12 df_xmr_weekly = spark.read.parquet(input_s3_path_xmr_weekly)
13 # Google Trends dataset for Monero
14 df_xmrggl = spark.read.parquet(input_s3_path_trends)
15
16 # Select only the "Settimana" and "Monero_interesse" columns, renaming them
17 df_xmrggl_selected = df_xmrggl.select(
18     col("Settimana").alias("Week_Start"),
19     col("Monero_interesse").alias("IndiceGoogleTrend")
20 )
21
22 # Perform join on weekly fields
23 # Week_Start is already aligned between the two dataframes
24 df_joined = df_xmr_weekly.join(df_xmrggl_selected, on="Week_Start", how="inner")
25
26 # Rename columns for the required format
27 df_final = df_joined.select(
28     col("Week_Start").alias("Data"),
29     col("Weekly_Avg_Price").alias("Prezzo"),
30     col("IndiceGoogleTrend")
31 )
32
33 # Save the final result as Parquet
34 df_final.write.mode("overwrite").parquet(output_s3_path_joined)
35
36 print("Parquet successfully created in bucketgolden.")
37
```

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPUs)	Worker typ
Succeeded	0	02/11/2025 16:57:12	02/11/2025 16:58:38	1 m 18 s	10 DPUs	G.1X

[Run details](#) | [Input arguments \(11\)](#) | [Continuous logs](#) | [Run insights](#) | [Metrics](#) | [Troubleshooting analysis - preview](#) | [Spark UI](#)

Job name	Start time (Local)	Glue version	Last modified on (Local)
XMR_Join_Price_Google trend	02/11/2025 16:57:12	5.0	02/11/2025 16:58:38

For the transformations, we have created two Glue jobs for each cryptocurrency:

AWS Glue Studio Info

Create job Info

Author in a visual interface focused on data flow. **Visual ETL**

Author using an interactive code notebook. **Notebook**

► **Example jobs** Info

Your jobs (4) Info

Filter jobs by property

☐	Job name	Type	Created by
☐	XMR_Join_Price_Google trend	Glue ETL	Script
☐	BTC_Join_Price_Google trend	Glue ETL	Script
☐	Modify XMR Price/Google Trends Files	Glue ETL	Script
☐	Modify BTC Price/Google Trends Files	Glue ETL	Script

The Parquet files resulting from the transformations have been saved in the final bucket, called **"bucketgolden"**:

bucketgolden Info

Oggetti | Proprietà | Autorizzazioni | Parametri | Gestione | P

Oggetti (2)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

☐	Nome	Tipo
☐	BTC_GoogleTrend_Joined.parquet/	Cartella
☐	XMR_GoogleTrend_Joined.parquet/	Cartella

BTC_GoogleTrend_Joined.parquet/

Oggetti | Proprietà

Oggetti (1)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

☐	Nome	Tipo
☐	part-00000-987ca16d-ddfb-4e77-a565-3462441e4ae1-c000.snappy.parquet	parquet

XMR_GoogleTrend_Joined.parquet/

Oggetti | Proprietà

Oggetti (1)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

☐	Nome	Tipo
☐	part-00000-4b60c8ae-cb63-4b0e-a752-e9108d73e5e1-c000.snappy.parquet	parquet

We visualize the Parquet files resulting from the union of the datasets.

Bitcoin: *Date* (month/day/year) – *Price* (weekly average) – *Sentiment through Google Searches* (Google Trends, values from 0 to 100).

Data	Prezzo	IndiceGoogleTrend
3/17/2019,	3,516.286	9
3/24/2019	3,556.571	9
3/31/2019	4,214.143	15
4/7/2019,	4,594	13
4/14/2019	4,629.857	11
4/21/2019	4,745.857	12
4/28/2019	4,816.429	11
5/5/2019,	5,440.714	15
5/12/2019	6,822.286	24
5/19/2019,	7,126.571	17
5/26/2019	7,718.714	19
6/2/2019,	7,118.286	16
6/9/2019,	7,296	14
6/16/2019	8,496.429	21
6/23/2019	10,248.571	34
6/30/2019	9,819.571	22
7/7/2019,	10,554	21
7/14/2019,	9,229.857	20

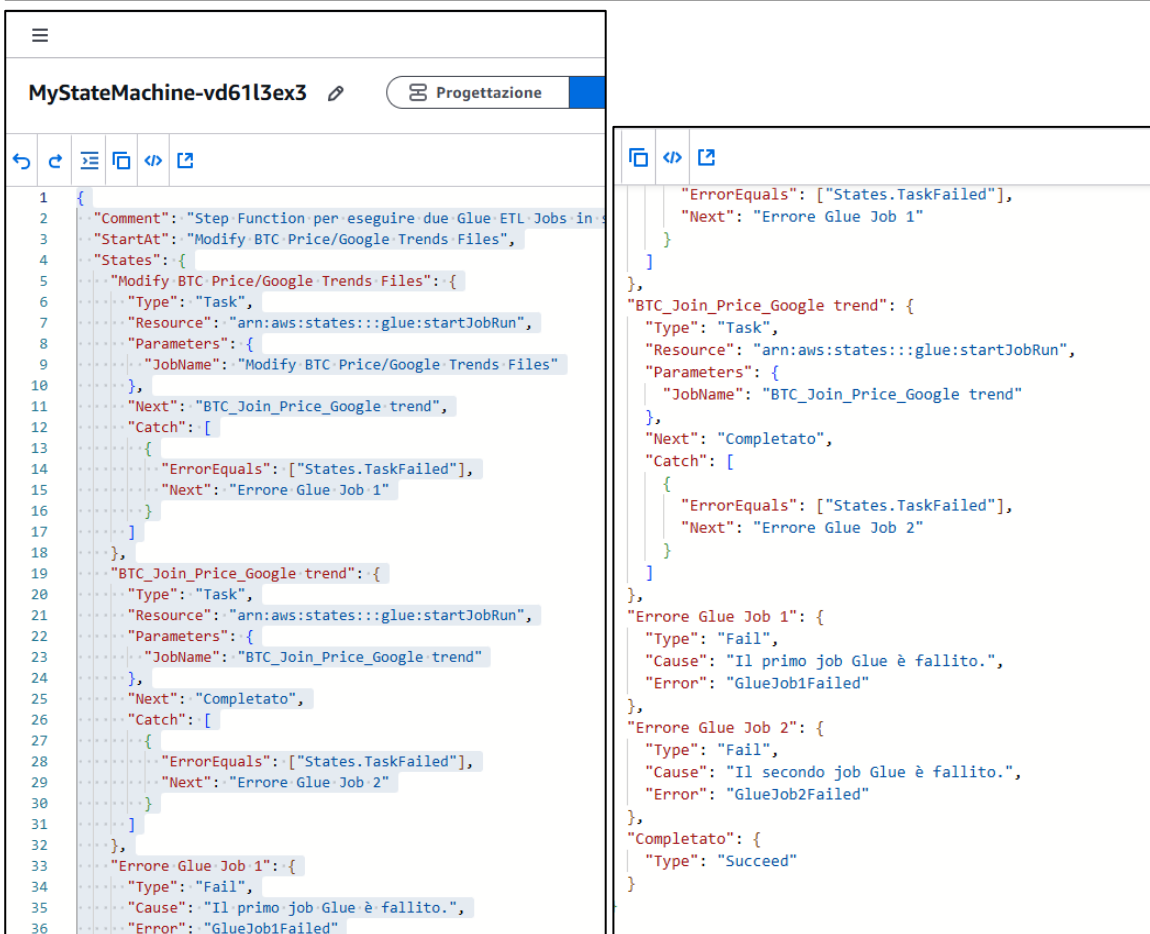
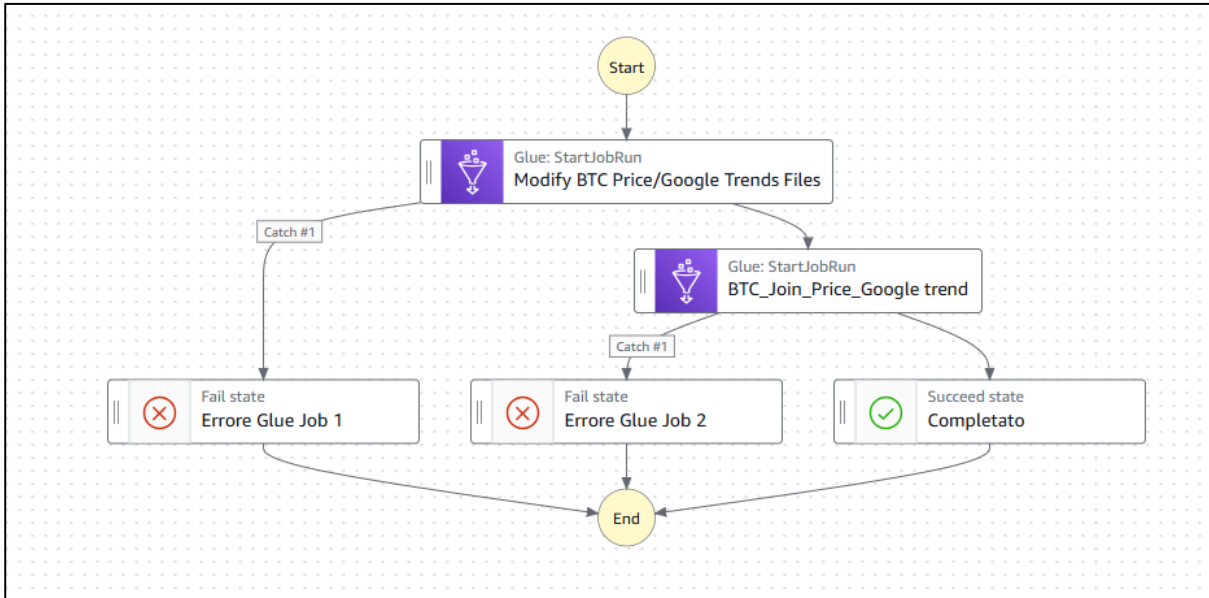
Monero: *Date* (month/day/year) – *Price* (weekly average) – *Sentiment through Google Searches* (Google Trends, values from 0 to 100).

Data	Prezzo	IndiceGoogleTrend
3/17/2019,	45.857	26
3/24/2019	46.143	23
3/31/2019.	56.857	28
4/7/2019,	60.143	25
4/14/2019	59.714	23
4/21/2019	58	22
4/28/2019	56.429	24
5/5/2019,	60.429	25
5/12/2019.	73.714	31
5/19/2019.	77.143	25
5/26/2019.	83.714	34
6/2/2019,	77.571	31
6/9/2019,	78.857	30
6/16/2019.	91	37
6/23/2019.	92.714	45
6/30/2019	79	35
7/7/2019, ..	86.714	32

Step Function – Pipeline for process orchestration.

AWS Step Functions allows for the orchestration of workflows on AWS, defining a complete automation of the data transformation pipeline, from raw data collection to data cleaning and dataset acquisition. To this end, we use the Glue transformations we have created.

Bitcoin (BTC):



Macchine a stati (2)

Qualsiasi tipo

Nome	Tipo	Data di creazione	Stato
MyStateMachine-vd6113ex3	Standard	11 feb 2025, 21:22:26 (UTC+01:00)	Attivo

Esecuzione: 9986c25d-ca44-48b2-a5d2-b74117ff22c0

Dettagli

Input e output di esecuzione

Definizione

Stato dell'esecuzione

✓ Riuscito

▶ 1	ExecutionStarted	
▶ 2	TaskStateEntered	Modify BTC Price/Google Trends Files
▶ 3	TaskScheduled	Modify BTC Price/Google Trends Files
▶ 4	TaskStarted	Modify BTC Price/Google Trends Files
▶ 5	TaskSucceeded	Modify BTC Price/Google Trends Files
▶ 6	TaskStateExited	Modify BTC Price/Google Trends Files
▶ 7	TaskStateEntered	BTC_Join_Price_Google trend
▶ 8	TaskScheduled	BTC_Join_Price_Google trend
▶ 9	TaskStarted	BTC_Join_Price_Google trend
▶ 10	TaskSucceeded	BTC_Join_Price_Google trend
▶ 11	TaskStateExited	BTC_Join_Price_Google trend
▶ 12	SucceedStateEntered	Completato
▶ 13	SucceedStateExited	Completato
▶ 14	ExecutionSucceeded	

We compose a similar process using the Glue files for Monero.

MyStateMachine-eyhwwb5z0

Progettazione

Codice

Annulla
Ripeti
Formato
Copia
Comandi
Visualizza id

```

1
2
3 "Comment": "Step Function per eseguire due Glue ETL Jobs in sequenza con gestio
4 "StartAt": "Modify XMR Price/Google Trends Files",
5 "States": {
6   "Modify XMR Price/Google Trends Files": {
7     "Type": "Task",
8     "Resource": "arn:aws:states:::glue:startJobRun",
9     "Parameters": {
10      "JobName": "Modify XMR Price/Google Trends Files"
11    },
12     "Next": "XMR_Join_Price_Google trend",
13     "Catch": [
14       {
15         "ErrorEquals": [
16           "States.TaskFailed"
17         ],
18         "Next": "Errore Glue Job 1"
19       }
20     ],
21   },
22   "XMR_Join_Price_Google trend": {
23     "Type": "Task",
24     "Resource": "arn:aws:states:::glue:startJobRun",
25     "Parameters": {
26      "JobName": "XMR_Join_Price_Google trend"
27    },
28     "Next": "Completato",
29     "Catch": [
30       {
31         "ErrorEquals": [
32           "States.TaskFailed"
33         ],
34         "Next": "Errore Glue Job 2"
35       }
36     ]
37   }
38 }

```

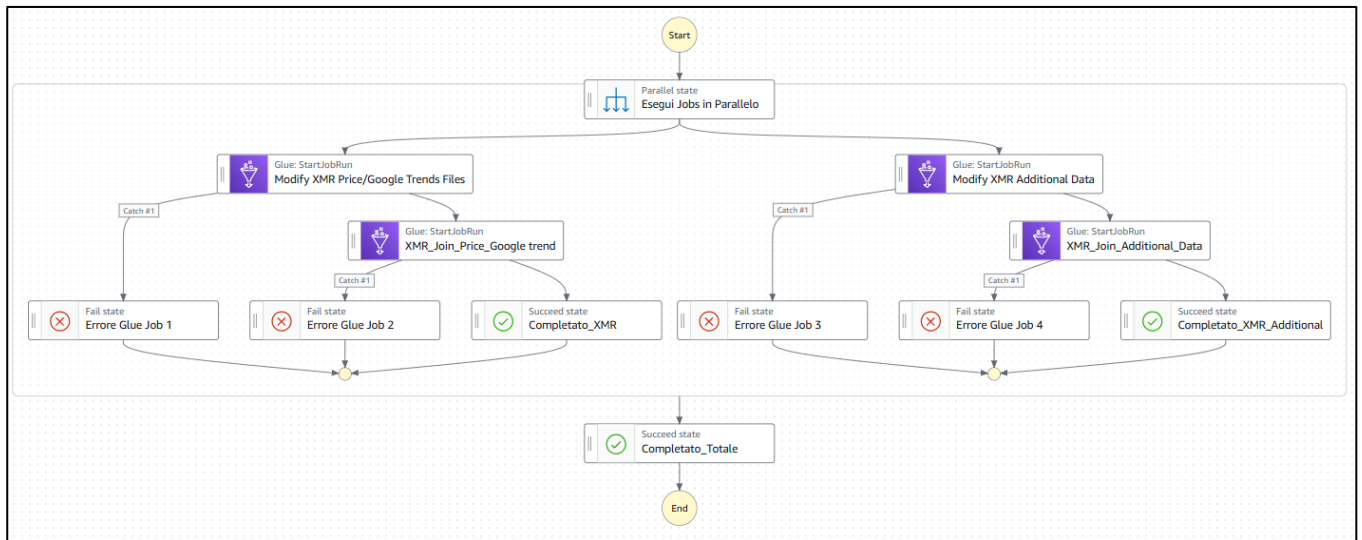
```

},
"Next": "Completato",
"Catch": [
  {
    "ErrorEquals": [
      "States.TaskFailed"
    ],
    "Next": "Errore Glue Job 2"
  }
],
"Errore Glue Job 1": {
  "Type": "Fail",
  "Cause": "Il primo job Glue è fallito.",
  "Error": "GlueJob1Failed"
},
"Errore Glue Job 2": {
  "Type": "Fail",
  "Cause": "Il secondo job Glue è fallito.",
  "Error": "GlueJob2Failed"
},
"Completato": {
  "Type": "Succeed"
}
}

```

Step Function for executing jobs in parallel

It is possible to run multiple state machines in parallel by using the Parallel state type. This feature improves efficiency and reduces execution time.



```
"Comment": "Step Function per eseguire due Glue ETL Jobs in parallelo",
"StartAt": "Esegui Jobs in Parallelo",
"States": {
  "Esegui Jobs in Parallelo": {
    "Type": "Parallel",
    "Branches": [
      {
        "StartAt": "Modify XMR Price/Google Trends Files",
        "States": {
          "Modify XMR Price/Google Trends Files": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun",
            "Parameters": {
              "JobName": "Modify XMR Price/Google Trends Files"
            },
            "Next": "XMR_Join_Price_Google trend",
            "Catch": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "Next": "Errore Glue Job 1"
              }
            ]
          },
          "XMR_Join_Price_Google trend": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun",
            "Parameters": {
              "JobName": "XMR_Join_Price_Google trend"
            },
            "Next": "Completato_XMR",
            "Catch": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "Next": "Errore Glue Job 2"
              }
            ]
          },
          "Errore Glue Job 1": {
            "Type": "Fail",
            "Cause": "Il primo job Glue è fallito.",
            "Error": "GlueJob1Failed"
          },
          "Errore Glue Job 2": {
            "Type": "Fail",
            "Cause": "Il secondo job Glue è fallito.",
            "Error": "GlueJob2Failed"
          },
          "Completato_XMR": {
            "Type": "Succeed"
          }
        ]
      },
      {
        "StartAt": "Modify XMR Additional Data",
        "States": {
          "Modify XMR Additional Data": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun",
            "Parameters": {
              "JobName": "Modify XMR Additional Data"
            },
            "Next": "XMR_Join_Additional_Data",
            "Catch": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "Next": "Errore Glue Job 3"
              }
            ]
          },
          "XMR_Join_Additional_Data": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun",
            "Parameters": {
              "JobName": "XMR_Join_Additional_Data"
            },
            "Next": "Completato_XMR_Additional",
            "Catch": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "Next": "Errore Glue Job 4"
              }
            ]
          },
          "Errore Glue Job 3": {
            "Type": "Fail",
            "Cause": "Il terzo job Glue è fallito.",
            "Error": "GlueJob3Failed"
          },
          "Errore Glue Job 4": {
            "Type": "Fail",
            "Cause": "Il quarto job Glue è fallito.",
            "Error": "GlueJob4Failed"
          },
          "Completato_XMR_Additional": {
            "Type": "Succeed"
          }
        ]
      }
    ]
  },
  "Completato_Totale": {
    "Type": "Succeed"
  }
}
```

```
},
"Next": "Completato_XMR_Additional",
"Catch": [
  {
    "ErrorEquals": [
      "States.TaskFailed"
    ],
    "Next": "Errore Glue Job 4"
  }
],
"Errore Glue Job 3": {
  "Type": "Fail",
  "Cause": "Il terzo job Glue è fallito.",
  "Error": "GlueJob3Failed"
},
"Errore Glue Job 4": {
  "Type": "Fail",
  "Cause": "Il quarto job Glue è fallito.",
  "Error": "GlueJob4Failed"
},
"Completato_XMR_Additional": {
  "Type": "Succeed"
},
"Next": "Completato_Totale"
},
"Completato_Totale": {
  "Type": "Succeed"
}
```

```
},
"Next": "Completato_XMR_Additional",
"Catch": [
  {
    "ErrorEquals": [
      "States.TaskFailed"
    ],
    "Next": "Errore Glue Job 4"
  }
],
"Errore Glue Job 3": {
  "Type": "Fail",
  "Cause": "Il terzo job Glue è fallito.",
  "Error": "GlueJob3Failed"
},
"Errore Glue Job 4": {
  "Type": "Fail",
  "Cause": "Il quarto job Glue è fallito.",
  "Error": "GlueJob4Failed"
},
"Completato_XMR_Additional": {
  "Type": "Succeed"
},
"Next": "Completato_Totale"
},
"Completato_Totale": {
  "Type": "Succeed"
}
```

Redshift

We complete the saving process of the tables in Redshift, making them available for analysis, query execution, and visualizations in environments such as AWS QuickSight.

Bitcoin (BTC):

The first screenshot shows a SQL query in the Redshift console to load data from an S3 bucket into a table named 'btc_googletrend_joined'. The query is as follows:

```
1 COPY dev.public.BTC_GoogleTrend_Joined
2 FROM 's3://bucketgolden/BTC_GoogleTrend_Joined.parquet/part-00000-987ca16d-ddfb-4e77-a565-3462441e4ae1-c000.snappy.parquet'
3 IAM_ROLE 'arn:aws:iam::423623841156:role/service-role/AmazonRedshift-CommandsAccessRole-...'
4 FORMAT AS PARQUET
```

The second screenshot shows the 'Summary' section of the console, indicating that the load operation was successful. It states: 'Load into table 'btc_googletrend_joined' completed, 260 record(s) loaded successfully.' It also shows 'Returned rows: 0', 'Query ID: 6082', and 'Elapsed time: 12.1s'.

The third screenshot shows a SQL query to create a new table named 'bitcoin' with the following schema:

```
1 CREATE TABLE bitcoin (
2   "Date" DATE PRIMARY KEY,      -- Colonna di tipo DATE, chiave primaria
3   Price FLOAT,                  -- Colonna per il prezzo, tipo FLOAT
4   IndiceGoogleTrend DECIMAL(10, 2) -- Colonna per l'indice di Google Trend, tipo DECIMAL
5 );
```

The 'Summary' section for this query shows 'Returned rows: 0', 'Elapsed time: 392ms', and 'Result set query:'.

The fourth screenshot shows a SQL query to inspect the columns of the 'btc_googletrend_joined' table:

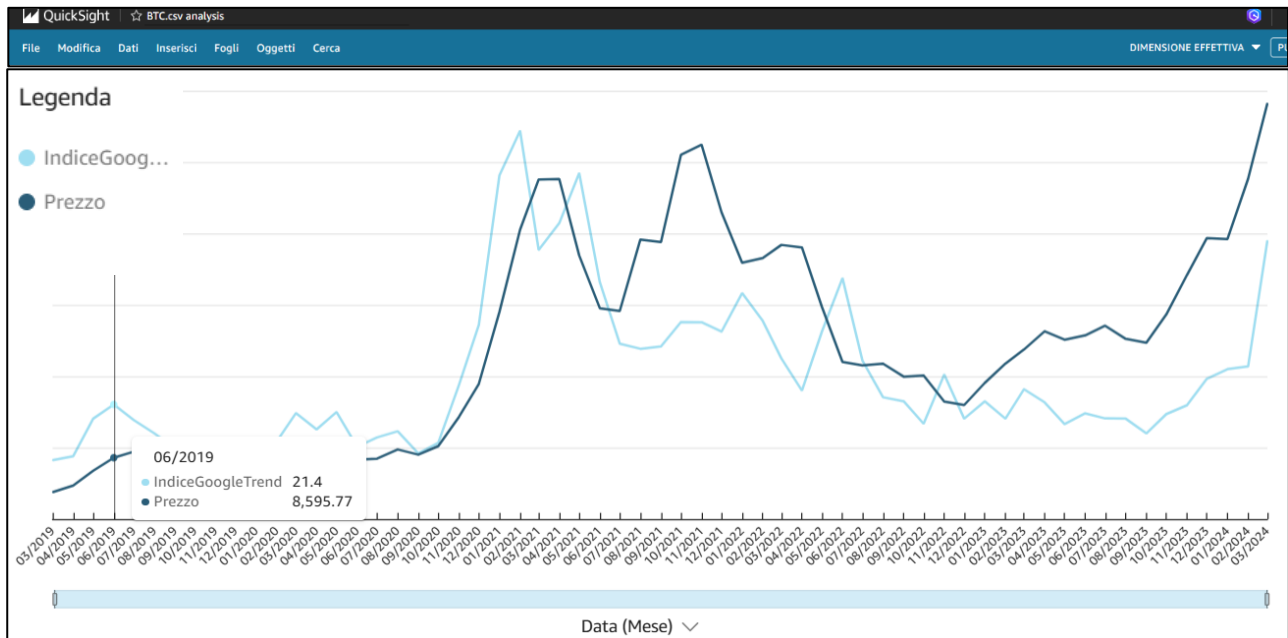
```
1 SELECT column_name, data_type
2 FROM information_schema.columns
3 WHERE table_name = 'btc_googletrend_joined';
4
```

I perform the same operations for the Monero file.

QuickSight

Once the final dataset is processed, it can be used to create visualizations that highlight trends over time, providing meaningful insights.

Bitcoin (BTC) Chart:



Insights: The two trends show a strong correlation throughout the entire performance period. In particular, a historic peak is recorded starting from September 2020. After that, a clear discontinuity is observed in the average values of the trends compared to the previous period. The search trend seems to accurately anticipate the immediate price movement of Bitcoin and has long-lasting effects on the price, keeping it at a significantly higher average compared to the period before September 2020.

From mid-2022, Bitcoin's price begins to become independent, even taking on a leading role in the relationship, possibly driven by other market and contextual factors, with the search trend following it. In a broader context, the sustained growth of online interest up to that point still appears to support the price, showing how the Google search variable continues to be a crucial factor in predicting Bitcoin's price movement.

Monero (XMR) Chart: The correlation between the search trend (light blue) and the price trend (blue) is quite clear and linear for the entire observed period.

